

DAX EX 1

Power BI Data Analysis Expressions (DAX)

Course:	Topic:	Estimated Time:	Level:
Data Analytics	DAX Measures & Calculated Columns	75–90 minutes	Intermediate

Overview

In this hands-on exercise you will build DAX measures and calculated columns in Power BI Desktop using a two-table sales model. You will progress from basic aggregations through filter context manipulation, time intelligence, and conditional logic — the core skills needed for real-world BI development.

You will work with two data files: SalesTransactions_DAX.csv (100 order rows, Jan–Dec 2024) and Products_DAX.csv (6 products with cost and list price). A relationship between the two tables will let you use RELATED and cross-table calculations.

Learning Objectives

By the end of this exercise, you will be able to:

- Distinguish between a DAX measure and a calculated column, and choose the right one for each scenario
- Write basic aggregation measures: SUM, COUNT, AVERAGE, DISTINCTCOUNT
- Create calculated columns using RELATED to bring data from a related table
- Use CALCULATE to override or modify filter context
- Apply DIVIDE for safe division that avoids divide-by-zero errors
- Write time intelligence measures using TOTALYTD and SAMEPERIODLASTYEAR
- Use IF and SWITCH for conditional logic in DAX
- Use ALL inside CALCULATE to remove filters for percentage-of-total calculations

Dataset Description

Download both files and load them into Power BI before starting the tasks.

Table 1: SalesTransactions_DAX.csv

Column	Type	Description
OrderID	Whole Number	Unique order identifier
OrderDate	Date	Transaction date (Jan–Dec 2024)
CustomerID	Text	Customer code (C001–C010)
CustomerName	Text	Full customer name

ProductID	Text	Foreign key — joins to Products_DAX.csv
Quantity	Whole Number	Units sold per order
UnitPrice	Decimal	Selling price per unit
Discount	Decimal	Discount rate (0.00–0.10)
Region	Text	Sales region: North, South, East, West
SalesRep	Text	Name of the sales representative

Table 2: Products_DAX.csv

Column	Type	Description
ProductID	Text	Primary key — links to SalesTransactions
ProductName	Text	Descriptive product name
Category	Text	Electronics, Accessories, or Storage
UnitCost	Decimal	Cost to the company per unit
ListPrice	Decimal	Standard selling price per unit

Setup Instructions

Complete these setup steps before working on the exercise tasks.

Load the Data

1. Open Power BI Desktop.
2. On the Home ribbon click Get Data → Text/CSV. Load SalesTransactions_DAX.csv, then repeat to load Products_DAX.csv. Click Transform Data for each to verify column types, then click Close & Apply.
3. In the Data pane confirm you have two tables: SalesTransactions and Products.

Create the Relationship

4. Click the Model view icon (the diagram icon on the left sidebar).
5. Drag ProductID from the SalesTransactions table and drop it onto ProductID in the Products table. Power BI creates a Many-to-One relationship (many orders → one product).
6. Confirm the relationship arrow shows * (many) on the SalesTransactions side and 1 (one) on the Products side.

Your model now has a star schema: SalesTransactions is the fact table and Products is the dimension table. All RELATED function calls in the exercise rely on this relationship.

Exercise Tasks

Task 1 — Basic Aggregation Measures

In Power BI, go to the Report view. Select the SalesTransactions table in the Data pane. Create each of the following measures using New Measure on the Home ribbon.

1. Create a measure called Total Revenue using the formula below:

```
Total Revenue =  
    SUMX(  
        SalesTransactions,  
        SalesTransactions[Quantity]  
            * SalesTransactions[UnitPrice]  
            * (1 - SalesTransactions[Discount])  
    )
```

2. Create a measure called Total Orders:

```
Total Orders = COUNTROWS(SalesTransactions)
```

3. Create a measure called Total Units Sold:

```
Total Units Sold = SUM(SalesTransactions[Quantity])
```

4. Create a measure called Average Order Value:

```
Average Order Value = DIVIDE([Total Revenue], [Total Orders])
```

5. Create a measure called Unique Customers:

```
Unique Customers = DISTINCTCOUNT(SalesTransactions[CustomerID])
```

6. Add a Card visual for each measure to verify the results. Your Total Revenue should be approximately \$475,000.

Tip: Use SUMX instead of SUM here because the revenue calculation requires a row-by-row expression (multiplying across three columns). SUM works only on a single column.

Task 2 — Calculated Columns with RELATED

Calculated columns are computed once at data refresh and stored in the table. They are useful for row-level values. Select the SalesTransactions table, then on the Table Tools ribbon click New Column.

1. Add a calculated column called Line Revenue (revenue for each individual order row):

```
Line Revenue =
    SalesTransactions[Quantity]
        * SalesTransactions[UnitPrice]
        * (1 - SalesTransactions[Discount])
```

2. Add a calculated column called Product Cost using RELATED to pull UnitCost from the Products table:

```
Product Cost =
    SalesTransactions[Quantity]
        * RELATED(Products[UnitCost])
```

3. Add a calculated column called Gross Profit:

```
Gross Profit =
    SalesTransactions[Line Revenue] - SalesTransactions[Product Cost]
```

4. Add a calculated column called Profit Margin % (as a decimal):

```
Profit Margin % =
    DIVIDE(
        SalesTransactions[Gross Profit],
        SalesTransactions[Line Revenue]
    )
```

5. Add a calculated column called Product Category using RELATED:

```
Product Category =
    RELATED(Products[Category])
```

6. Switch to the Data view and inspect the SalesTransactions table. Verify that Line Revenue, Product Cost, Gross Profit, Profit Margin %, and Product Category columns all appear and contain values.

RELATED works because of the relationship you created in Setup. It navigates from the many side (SalesTransactions) to the one side (Products) and returns the value in the specified column for each row.

Task 3 — CALCULATE: Filtering Measures

CALCULATE is the most powerful function in DAX. It evaluates an expression in a modified filter context. Create these measures in the SalesTransactions table.

1. Create a measure for revenue from the North region only:

```
Revenue North =
    CALCULATE(
        [Total Revenue],
        SalesTransactions[Region] = "North"
```

```
)
```

2. Create a similar measure for the South region:

```
Revenue South =  
    CALCULATE(  
        [Total Revenue],  
        SalesTransactions[Region] = "South"  
    )
```

3. Create a measure for Electronics revenue only using the Product Category calculated column:

```
Revenue Electronics =  
    CALCULATE(  
        [Total Revenue],  
        SalesTransactions[Product Category] = "Electronics"  
    )
```

4. Create a measure for the percentage of total revenue each region represents. The ALL function removes any region filter so the denominator is always the grand total:

```
Revenue % of Total =  
    DIVIDE(  
        [Total Revenue],  
        CALCULATE([Total Revenue], ALL(SalesTransactions[Region]))  
    )
```

5. Add a Matrix visual to the report canvas. Put Region in Rows, and drag Total Revenue, Revenue % of Total, and Revenue Electronics into Values. Observe how Revenue % of Total changes by region.

Task 4 — Conditional Logic: IF and SWITCH

DAX supports row-level and measure-level conditional logic. These can be used in both calculated columns and measures.

1. Add a calculated column to SalesTransactions that classifies each order by deal size:

```
Deal Size =  
    SWITCH(  
        TRUE(),  
        SalesTransactions[Line Revenue] >= 3000, "Large",  
        SalesTransactions[Line Revenue] >= 1000, "Medium",  
        "Small"  
    )
```

2. Add a calculated column that flags orders where a discount was applied:

```
Discount Applied =  
    IF(SalesTransactions[Discount] > 0, "Yes", "No")
```

3. Create a measure that counts only Large deals:

```
Large Deal Count =  
    CALCULATE(  
        [Total Orders],  
        SalesTransactions[Deal Size] = "Large"  
    )
```

4. Create a measure for the percentage of orders that received a discount:

```
Discount Rate % =  
    DIVIDE(  
        CALCULATE([Total Orders], SalesTransactions[Discount Applied] =  
            "Yes"),  
        [Total Orders]  
    )
```

5. Build a stacked bar chart: X-axis = Region, Y-axis = Total Revenue, Legend = Deal Size. Observe how deal mix varies by region.

Task 5 — Time Intelligence

Time intelligence functions require a continuous date table marked as a Date Table. First create a Date Table using DAX, then build the measures.

1. Create a new calculated table called DateTable. On the Home ribbon click New Table:

```
DateTable =  
    ADDCOLUMNS(  
        CALENDAR (DATE (2024,1,1), DATE (2024,12,31)),  
        "Year",      YEAR ([Date]),  
        "Month",     MONTH ([Date]),  
        "MonthName", FORMAT ([Date], "MMMM"),  
        "Quarter",   "Q" & QUARTER ([Date]),  
        "Weekday",   FORMAT ([Date], "dddd")  
    )
```

2. Right-click DateTable in the Data pane and select Mark as Date Table. Set the Date Column to Date.
3. In Model view, create a relationship between DateTable[Date] and SalesTransactions[OrderDate] (Many-to-One from SalesTransactions to DateTable).
4. Create a Year-to-Date revenue measure:

```
Revenue YTD =  
    TOTALYTD(  
        [Total Revenue],  
        DateTable[Date]  
    )
```

5. Create a measure for revenue in the same period last year (useful for year-over-year comparison):

```
Revenue LY =  
    CALCULATE(  
        [Total Revenue],  
        SAMEPERIODLASTYEAR(DateTable[Date])  
    )
```

6. Create a Month-over-Month change measure:

```
Revenue MoM Change =  
    [Total Revenue]  
    - CALCULATE(  
        [Total Revenue],  
        DATEADD(DateTable[Date], -1, MONTH)  
    )
```

7. Add a Line Chart: X-axis = DateTable[MonthName], Y-axis = Total Revenue and Revenue YTD. Confirm Revenue YTD is cumulative and always >= Total Revenue for any given month.

Time intelligence functions only work correctly when: (1) your date column has no gaps, (2) the table is marked as a Date Table, and (3) the relationship between the date table and the fact table exists.

Task 6 — Profitability Analysis Measures

Using the calculated columns from Task 2 and your existing measures, build a complete profitability analysis.

1. Create a Total Cost measure:

```
Total Cost = SUM(SalesTransactions[Product Cost])
```

2. Create a Total Gross Profit measure:

```
Total Gross Profit = SUM(SalesTransactions[Gross Profit])
```

3. Create an Overall Profit Margin % measure:

```
Overall Profit Margin % =  
    DIVIDE([Total Gross Profit], [Total Revenue])
```

4. Create a measure for the highest-grossing product by profit using MAXX:

```
Max Product Profit =  
    MAXX(  
        VALUES(SalesTransactions[ProductID]),  
        [Total Gross Profit]
```

)

- Build a clustered bar chart: Y-axis = Products[ProductName], X-axis = Total Revenue and Total Gross Profit. Add Overall Profit Margin % as a tooltip.
- Add a table visual with columns: Products[Category], Total Revenue, Total Cost, Total Gross Profit, Overall Profit Margin %. Format Overall Profit Margin % as a percentage in the Format pane.

Reference

DAX Function Quick Reference

Function	Category	Description & Syntax
<code>SUM(column)</code>	Aggregation	Adds all values in a column. <code>SUM(Table[Column])</code>
<code>SUMX(table, expr)</code>	Aggregation	Row-by-row sum of an expression. Use when calculation needs multiple columns.
<code>COUNT(column)</code>	Aggregation	Counts non-blank values in a column.
<code>COUNTROWS(table)</code>	Aggregation	Counts all rows in a table (including related filtered tables).
<code>AVERAGE(column)</code>	Aggregation	Returns the arithmetic mean of a column.
<code>DISTINCTCOUNT(column)</code>	Aggregation	Counts unique values. Useful for customer or product counts.
<code>MAXX(table, expr)</code>	Aggregation	Returns the maximum value of an expression evaluated per row.
<code>CALCULATE(expr, filter1, ...)</code>	Filter	Evaluates an expression after applying one or more filter modifications. The most important DAX function.
<code>ALL(table or column)</code>	Filter	Removes all filters from a table or column. Used inside <code>CALCULATE</code> .
<code>FILTER(table, condition)</code>	Filter	Returns a filtered table. Often used as a filter argument inside <code>CALCULATE</code> .
<code>RELATED(column)</code>	Relationship	Returns a value from a related table (many-to-one direction).
<code>RELATEDTABLE(table)</code>	Relationship	Returns a table of rows related to the current row (one-to-many direction).
<code>DIVIDE(num, denom, alt)</code>	Math	Safe division. Returns alt (default BLANK) if denominator is 0 or BLANK.
<code>IF(condition, true, false)</code>	Logical	Returns one of two values based on a condition.
<code>SWITCH(expr, val1, res1, ...)</code>	Logical	Multi-branch conditional. <code>SWITCH(TRUE(), ...)</code> evaluates conditions in order.
<code>TOTALYTD(expr, dates)</code>	Time Intel.	Cumulative total from the beginning of the year to the current date.
<code>SAMEPERIODLASTYEAR(dates)</code>	Time Intel.	Returns a date table shifted back exactly one year. Used inside <code>CALCULATE</code> .
<code>DATEADD(dates, n, interval)</code>	Time Intel.	Shifts a set of dates by n intervals (DAY, MONTH, QUARTER, YEAR).

<code>CALENDAR(start, end)</code>	Table	Creates a single-column table of dates from start to end.
<code>ADDCOLUMNS(table, name, expr, ...)</code>	Table	Adds computed columns to an existing or generated table.
<code>VALUES(column)</code>	Table	Returns the distinct values of a column as a table. Respects current filter context.
<code>FORMAT(value, format)</code>	Text	Converts a value to text using a format string. E.g., <code>FORMAT([Date], "MMMM")</code> .

Measure vs. Calculated Column

	Measure	Calculated Column
When calculated	At query time, in response to visuals/filters	Once at data refresh, stored in the model
Stored in model	No (computed on demand)	Yes (adds a column to the table)
Use for	Aggregations, KPIs, context-dependent values	Row-level attributes, values used in slicers
Can use RELATED	Yes	Yes
Appears in slicer	No	Yes

Understanding Filter Context

Every DAX measure is evaluated inside a filter context — the set of active filters applied by slicers, visuals, and row headers on the report canvas. `CALCULATE` is the only function that can modify the filter context.

Example: If a bar chart has Region on the X-axis, each bar evaluates [Total Revenue] inside the filter context `Region = "North"` (or South, East, West). Adding `ALL(SalesTransactions[Region])` inside `CALCULATE` removes that filter, giving the same grand total for every bar.

Recommended Resources

- Microsoft Learn — DAX Overview:
<https://learn.microsoft.com/en-us/dax/dax-overview>
- DAX Guide (daxguide.com) — function reference with interactive examples:
<https://daxguide>
- SQLBI — articles and videos on DAX patterns:
<https://www.sqlbi.com/articles/>

Reflections

Answer each question in your own words after completing the exercise. These responses build conceptual understanding — not just formula recall.

Section A — Measures and Calculated Columns

1. Why did we use SUMX for Total Revenue instead of just SUM?

Describe the difference between row context and filter context in your answer.

2. When should you use a calculated column instead of a measure?

Give one example from this exercise where a calculated column was the right choice, and explain why a measure would not have worked in that specific scenario.

Section B — CALCULATE and Filter Context

3. Explain what CALCULATE does in your own words.

Use the Revenue North measure as your example. What context does it evaluate [Total Revenue] in, and how is that different from just writing [Total Revenue] in a visual filtered to North?

4. What does ALL do inside CALCULATE, and why is it needed for Revenue % of Total?

What would happen to the Revenue % of Total measure if you removed ALL from the formula?

Section C — Time Intelligence

5. Why do time intelligence functions require a dedicated Date Table?

What is "date granularity continuity" and why does DAX require it for TOTALYTD and SAMEPERIODLASTYEAR to work correctly?

6. SAMEPERIODLASTYEAR returned BLANK for all months in this exercise. Why?

The dataset only covers 2024. What would you need to add to the dataset for SAMEPERIODLASTYEAR to return meaningful values, and how would you test it?

Section D — Challenge

7. Write a new measure that shows the Top 3 products by Total Revenue.

You will need to research TOPN and SUMX together. Write the formula and explain what each argument does.

Solution Guide

Review only after completing the exercise on your own. Reference values are approximate due to rounding.

Expected Measure Values (Card Visuals, No Filters Active)

Measure	Expected Value
Total Revenue	~\$474,820
Total Orders	100
Total Units Sold	1,035
Average Order Value	~\$4,748
Unique Customers	10
Total Cost	~\$239,125
Total Gross Profit	~\$235,695
Overall Profit Margin %	~49.6%

Task Solution Notes

Task 1 — Why SUMX?

SUM(SalesTransactions[UnitPrice]) would give you the raw sum of all unit prices, ignoring Quantity and Discount entirely. SUMX iterates each row, calculates Quantity * UnitPrice * (1 - Discount) for that row, and then sums all the row results. Any time revenue involves multiplication across columns, SUMX is the correct approach.

Task 2 — RELATED Requirement

RELATED requires an active relationship. If your relationship is inactive or the cardinality direction is reversed (one-to-many pointing the wrong way), RELATED will return BLANK for every row. Always verify the relationship arrow direction in Model view: the arrow should point from Products to SalesTransactions.

Task 3 — CALCULATE Filter Behavior

Revenue North with a Region slicer set to "South":

```
Revenue North =  
    CALCULATE([Total Revenue], SalesTransactions[Region] = "North")
```

The CALCULATE filter OVERRIDES the slicer filter for Region. Revenue North will always show North revenue regardless of what the slicer is set to. This is a common source of confusion: CALCULATE filter arguments replace (not add to) the existing filter on that column.

Task 3 — Revenue % of Total Explained

```
Revenue % of Total =
    DIVIDE(
        [Total Revenue],
        CALCULATE([Total Revenue],
            ALL(SalesTransactions[Region])
        )
    )

-- Result in a visual filtered to "North":
-- Numerator = North revenue (~$120,000)
-- Denominator = ALL revenue (~$474,820)
-- Result = ~25.3%
```

Task 4 — SWITCH(TRUE(), ...) Pattern

SWITCH(TRUE(), ...) is a common DAX idiom. It evaluates each condition in order and returns the result for the FIRST condition that is TRUE. This avoids deeply nested IF statements. The order matters: put the most restrictive condition first (≥ 3000 before ≥ 1000), otherwise every order above \$1,000 would match the second condition before reaching the third.

Task 5 — Time Intelligence Troubleshooting

If Revenue YTD returns the same value as Total Revenue for every month, the most likely causes are: (1) DateTable is not marked as a Date Table, (2) the relationship between DateTable[Date] and SalesTransactions[OrderDate] is missing or inactive, or (3) the OrderDate column has a text data type instead of Date. Verify all three before debugging the DAX formula.

Task 7 — Challenge Answer

```
Top 3 Revenue =
    SUMX(
        TOPN(
            3,
            VALUES(SalesTransactions[ProductID]),
            [Total Revenue],
            DESC
        ),
        [Total Revenue]
    )

-- return top 3 rows
-- from this list of
-- ordered by Total Revenue
-- highest first
-- sum Total Revenue for those 3
```

TOPN returns a table of the top N rows. SUMX then iterates that table and sums the expression. Note: this measure returns the combined revenue of the top 3 products — it does not return product names. To display a list, use a visual with ProductName and [Total Revenue] sorted descending, then use a Top N visual filter.

Reflection Reference Answers

1. SUMX vs SUM

SUM operates on a single column and cannot perform calculations across columns. SUMX creates a row context: it visits each row in the table, evaluates the expression (which can reference multiple columns in that row), and then aggregates the results. Revenue requires multiplying three columns together (Quantity, UnitPrice, and the discount factor) which is only possible with an iterator like SUMX.

2. Calculated Column vs. Measure

The Deal Size and Product Category columns are good examples. Deal Size is used in a slicer and as a Legend in a bar chart — both require the value to exist as a column in the table, not as a measure. A measure cannot appear in a slicer or legend because it produces a scalar value, not a per-row label. Similarly, Product Category pulled with RELATED needs to exist as a column so it can be used in slicers and as a CALCULATE filter argument.

3. What CALCULATE Does

CALCULATE evaluates its first argument (an expression or measure) inside a modified filter context. Revenue North hardcodes Region = "North" as a filter, meaning even if the visual or slicer has no region filter (or a different one), the measure always sees only North rows. Placing [Total Revenue] in a visual already filtered to North by a slicer achieves the same visual result, but CALCULATE makes the filter permanent and independent of what the slicer says.

4. ALL Inside CALCULATE

ALL removes the filter on Region from the evaluation context. Without ALL, the denominator in Revenue % of Total would equal the numerator (both filtered to the same region by the visual), and every region would show 100%. ALL forces the denominator to always be the grand total across all regions, making the percentage meaningful.

5. Why Date Table is Required

Time intelligence functions need a contiguous, gap-free sequence of dates to perform period calculations correctly. If the fact table has dates for only 80 of 365 days, TOTALYTD cannot know that the missing days should contribute zero — it would skip them. A dedicated Date Table provides every single date in the range, ensuring correct cumulative and comparison calculations. The "Mark as Date Table" setting tells the DAX engine to use this table as the authoritative time spine.

6. SAMEPERIODLASTYEAR Returns BLANK

SAMEPERIODLASTYEAR shifts the date context back by exactly one year. Since the dataset only has 2024 data, the function looks for 2023 dates, finds none, and returns BLANK. To test it properly, you would add 2023 transaction rows to SalesTransactions_DAX.csv and extend the DateTable CALENDAR to DATE(2023,1,1). Then the 2024 measure would show 2023 values side by side for year-over-year comparison.